

A Technical Introduction to Backpropagation

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

Section 1

For a training set there will be a set of input–output pairs, $\{(x_i, y_i)\}$. For each input–output pair (x_i, y_i) in the training set, the loss of the model on that pair is the cost of the difference between the predicted output $g(x_i)$ and the target output y_i :

Section 2

Therefore, the derivative with respect to o_j can be calculated if all the derivatives with respect to the outputs o_{ℓ} of the next layer – the ones closer to the output neuron – are known. [Note, if any of the neurons in set L were not connected to neuron j , they would be independent of w_{ij} and the corresponding partial derivative under the summation would vanish to 0.] Substituting Eq. 2, Eq. 3 Eq.4 and Eq. 5 in Eq. 1 we obtain:

Section 3

Learning as an optimization problem To understand the mathematical derivation of the backpropagation algorithm, it helps to first develop some intuition about the relationship between the actual output of a neuron and the correct output for a particular training example. Consider a simple neural network with two input units, one output unit and no hidden units, and in which each neuron uses a linear output (unlike most work on neural networks, in which mapping from inputs to outputs is non-linear) that is the weighted sum of its input. Initially, before training, the weights will be set randomly. Then the neuron learns from training examples, which in this case consist of a set of tuples (x_1, x_2, t) where x_1 and x_2 are the inputs to the network and t is the correct output (the output the network should produce given those inputs, when it has been trained). The initial network, given x_1 and x_2 , will compute an output y that likely differs from t (given random weights). A loss function $L(t, y)$ is used for measuring

the discrepancy between the target output t and the computed output y . For regression analysis problems the squared error can be used as a loss function, for classification the categorical cross-entropy can be used. As an example consider a regression problem using the square error as a loss:

Section 4

Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum; also, it has trouble crossing plateaus in the error function landscape. This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback, but Yann LeCun et al. argue that in many practical problems, it is not. Backpropagation learning does not require normalization of input vectors; however, normalization could improve performance. Backpropagation requires the derivatives of activation functions to be known at network design time.

Section 5

Assumptions The mathematical expression of the loss function must fulfill two conditions in order for it to be possibly used in backpropagation. The first is that it can be written as an average $E = \frac{1}{n} \sum_x E_x$ over error functions E_x , for n individual training examples, x . The reason for this assumption is that the backpropagation algorithm calculates the gradient of the error function for a single training example, which needs to be generalized to the overall error function. The second assumption is that it can be written as a function of the outputs from the neural network.

Section 6

However, if j is in an arbitrary inner layer of the network, finding the derivative E with respect to o_j is less obvious. Considering E as a function with the inputs being all neurons $L = \{u, v, \dots, w\}$ receiving input from neuron j ,

Section 7

The input net j to a neuron is the weighted sum of outputs o_k of previous neurons. If the neuron is in the first layer after the input layer, the o_k of the input layer are simply the inputs x_k to the network. The number of input units to the neuron is n . The variable w_{kj} denotes the weight between neuron k of the previous layer and neuron j of the current layer.

Section 8

To compute this, one starts with the input x and works forward; denote the weighted input of each hidden layer as z^l and the output of hidden layer l as the activation a^l . For backpropagation, the activation a^l as well as the derivatives $(f^l)'$ (evaluated at z^l) must be cached for use during the backwards pass. The derivative of the loss in terms of the inputs is given by the chain rule; note that each term is a total derivative, evaluated at the value of the network (at each node) on the input x :

Section 9

This is the reason why backpropagation requires that the activation function be differentiable. (Nevertheless, the ReLU activation function, which is non-differentiable at 0, has become quite popular, e.g. in AlexNet) The first factor is straightforward to evaluate if the neuron is in the output layer, because then $o_j = y$ and

Section 10

Motivation The goal of any supervised learning algorithm is to find a function that best maps a set of inputs to their correct output. The motivation for backpropagation is to train a multi-layered neural network such that it can learn the appropriate internal representations to allow it to learn any arbitrary mapping of input to output.